智元D1开发教程 | 强化学习怎么验证？Sim2Sim框架精讲！

原创 敢敢不敢写bug 敢敢AUTOHUB 2026年4月9日 08:45 上海 听全文

智元D1开发教程 | 强化学习怎么验证？Sim2Sim框架精讲！

1. 前言

在现代机器人技术的发展进程中，强化学习（Reinforcement Learning, RL）已经成为实现机器人自主运动控制的核心技术之一。传统的机器人控制方法依赖于精确的动力学建模和手工设计的控制器，这种方法在面对复杂地形和动态环境时往往表现出较大的局限性。而基于强化学习的控制方法通过让机器人在仿真环境中进行大量试错学习，能够自动发现最优的运动策略，从而实现更加鲁棒和自然的运动行为。

然而，**从仿真环境到真实机器人的部署（Sim-to-Real Transfer）一直是强化学习应用于机器人领域的主要挑战之一。仿真环境与真实世界之间存在的差异（Reality Gap）可能导致在仿真中表现良好的策略在实际部署时失效**。为了解决这一问题，研究者们开发了各种技术，包括域随机化（Domain Randomization）、系统辨识（System Identification）以及更精确的仿真建模等方法。本文将深入介绍开源的rl_sar框架，这是一个专门为四足机器人、轮足机器人和人形机器人设计的强化学习算法仿真验证与实物部署框架。

2. 框架概述与设计理念

2.1 项目定位



自媒体人的数字员工，从选题到发布全链路开干。
DClaw-开箱即用的AI助手
了解更多

rl_sar框架的核心定位是作为强化学习策略从训练到部署的桥梁。在典型的机器人强化学习工作流程中，研究者通常使用IsaacGym或IsaacSim等高性能仿真器进行策略训练，训练完成后需要将策略部署到Gazebo等物理仿真器中进行验证，最终部署到真实机器人上。rl_sar框架正是为了简化这一流程而设计的，它提供了统一的接口来处理不同仿真环境和真实机器人之间的差异。

2.2 系统架构总览

下图展示了rl_sar框架的整体系统架构：



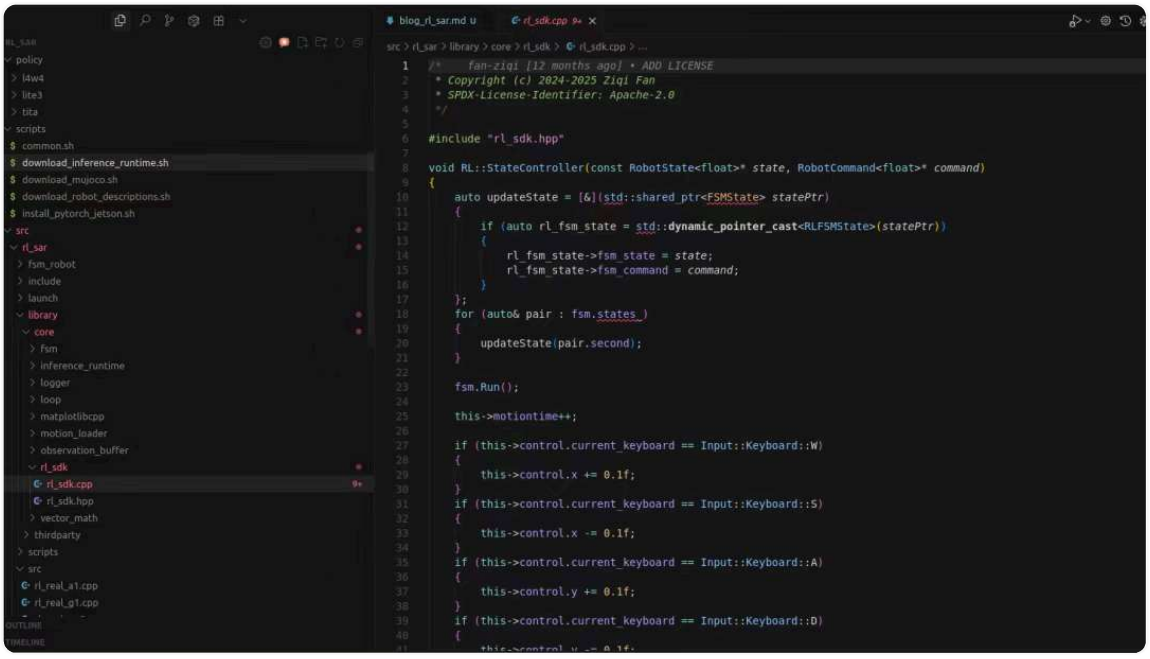
框架采用分层设计，主要包含以下几个层次：

1. **仿真/实机层**：负责与具体的仿真环境或真实机器人硬件进行交互
2. **核心库层（rl_sdk）**：提供强化学习控制的核心功能，包括观测计算、模型推理、输出计算等
3. **策略配置层**：管理机器人参数和强化学习策略的配置文件
4. **输入接口层**：处理键盘、手柄和ROS话题等多种输入方式

这种分层设计使得框架具有良好的可扩展性，开发者可以方便地添加新的机器人平台或仿真环境，而无需修改核心控制逻辑。

3. 核心模块深度分析

rl_sar框架的核心代码位于`src/rl_sar/library/core/`目录下，包含多个精心设计的模块。理解这些模块的设计原理和实现细节，是掌握整个框架的关键。本节将从架构设计的角度介绍各个核心模块的功能和相互关系。



3.1 RL SDK核心类（rl_sdk）

RL SDK是整个框架的基础，定义了强化学习控制器的核心数据结构和接口。框架使用模板化的数据结构来表示机器人的状态和控制命令，主要包含`RobotState`和`RobotCommand`两个核心结构体。`RobotState`封装了IMU数据（四元数、角速度、加速度）和电机状态（关节位置、速度、力矩估计等），而`RobotCommand`则包含了完整的PD控制参数（目标位置、目标速度、前馈力矩、位置增益kp、速度增益kd）。这种设计将电机的位置控制参数完整地封装在命令结构中，使得控制器可以灵活地调整每个关节的控制特性。

```cpp
// 机器人控制命令结构体
template <typename T>
struct RobotCommand
{
    struct MotorCommand
    {
        std::vector<int> mode;     // 电机控制模式
        std::vector<T> q;          // 目标关节位置
        std::vector<T> dq;         // 目标关节速度
        std::vector<T> tau;        // 前馈力矩
        std::vector<T> kp;         // 位置增益
        std::vector<T> kd;         // 速度增益
    } motor_command;
};

// 机器人状态结构体
template <typename T>
struct RobotState
```

```
{
    struct IMU
    {
        std::vector<T> quaternion = {1.0f, 0.0f, 0.0f, 0.0f}; // 四元数 (w, x, y, z)
        std::vector<T> gyroscope = {0.0f, 0.0f, 0.0f};        // 角速度
        std::vector<T> accelerometer = {0.0f, 0.0f, 0.0f};    // 加速度
    } imu;

        {
            int sdk_leg = SDK_LEG_MAP[leg];  // 获取对应的SDK腿索引
            int base_idx = leg * 3;          // rl_sar中该腿的起始索引

            // 位置映射
            state->motor_state.q[base_idx + 0] = this->d1_motor_state->q_abad[sdk_leg
            state->motor_state.q[base_idx + 1] = this->d1_motor_state->q_hip[sdk_leg
            state->motor_state.q[base_idx + 2] = this->d1_motor_state->q_knee[sdk_leg

            // 速度映射
            state->motor_state.dq[base_idx + 0] = this->d1_motor_state->qd_abad[sdk_
            state->motor_state.dq[base_idx + 1] = this->d1_motor_state->qd_hip[sdk_le
    struct MotorState
    {
        std::vector<T> q;         // 当前关节位置
        std::vector<T> dq;        // 当前关节速度
        std::vector<T> ddq;       // 关节加速度
        std::vector<T> tau_est;   // 估计力矩
        std::vector<T> cur;       // 电流
    } motor_state;
};
```

观测数据结构`Observations`定义了强化学习策略的输入向量，包括线速度、角速度、重力向量、用户命令、基座姿态四元数、关节位置、关节速度以及上一时刻的动作。这些观测量的选择和组合方式需要与训练时保持完全一致，否则会导致策略在部署时表现异常。

```
template <typename T>
struct Observations
{
    std::vector<T> lin_vel;      // 线速度（通常来自状态估计）
    std::vector<T> ang_vel;      // 角速度（来自IMU陀螺仪）
    std::vector<T> gravity_vec;  // 重力向量（用于姿态感知）
    std::vector<T> commands;     // 用户命令（前进、侧移、转向）
    std::vector<T> base_quat;    // 基座四元数（姿态）
    std::vector<T> dof_pos;      // 关节位置
    std::vector<T> dof_vel;      // 关节速度
    std::vector<T> actions;      // 上一时刻的动作（用于时序一致性）
};
```

`RL`类是所有机器人控制器的基类，采用了模板方法设计模式。它定义了三个纯虚函数：`Forward()`负责模型前向推理，`GetState()`负责从硬件获取机器人状态，`SetCommand()`负责将控制命令发送给机器人。核心的控制逻辑（如观测计算、输出计算、状态管理）在基类中统一实现，而与具体硬件相关的操作则由子类根据不同的

机器人平台分别实现。这种设计使得添加新机器人时只需要实现这三个接口函数即可。

```
class RL
{
public:
    // 纯虚函数 - 必须由子类实现
    virtual std::vector<float> Forward()= 0;                                  // 模型前向推理
    virtual void GetState(RobotState<float> *state)= 0;                       // 获取机器人状态
    virtual void SetCommand(const RobotCommand<float> *command)= 0;    // 发送控制命令

    // 核心功能函数
    void InitRL(std::string robot_config_path);           // 初始化RL系统
    std::vector<float> ComputeObservation();              // 计算观测向量
    void ComputeOutput(...);                              // 计算控制输出
    void StateController(...);                            // 状态控制器（FSM驱动）

protected:
    YamlParams params;                    // YAML配置参数
    Observations<float> obs;              // 当前观测
    RobotState<float> robot_state;        // 机器人状态
    RobotCommand<float> robot_command;    // 控制命令
    FSM fsm;                              // 有限状态机
    std::unique_ptr<InferenceRuntime::Model> model;  // 推理模型
    ObservationBuffer history_obs_buf;    // 历史观测缓冲区
};
```

3.2 推理运行时（Inference Runtime）

推理运行时模块负责加载和执行训练好的神经网络模型，是连接训练环境和部署环境的关键桥梁。框架采用策略模式设计了统一的模型接口`Model`，定义了`load()`、`is_loaded()`、`forward()`等标准方法。基于这个接口，框架实现了`TorchModel`和`ONNXModel`两个具体类，分别支持PyTorch TorchScript格式（.pt文件）和ONNX格式（.onnx文件）的模型。

```
namespace InferenceRuntime
{
    // 模型基类接口
    class Model
    {
    public:
        virtual bool load(const std::string& model_path)= 0;
        virtual bool is_loaded() const= 0;
        virtual std::vector<float> forward(const std::vector<std::vector<float>>& inp
        virtual std::string get_model_type() const= 0;
    };

    // TorchScript模型实现
    class TorchModel : public Model { /* ... */ };

    // ONNX Runtime模型实现
```

```
    class ONNXModel : public Model { /* ... */ };

    // 模型工厂类
    class ModelFactory
    {
    public:
        enum class ModelType { TORCH, ONNX, AUTO };
        static std::unique_ptr<Model> load_model(const std::string& model_path,
                                                 ModelType type = ModelType::AUTO);
    };
}
```

模型工厂类`ModelFactory`提供了便捷的模型加载功能，能够根据文件扩展名自动检测模型类型。这种设计使得用户可以根据部署环境的需求选择合适的推理后端。例如，在资源受限的嵌入式平台上，ONNX Runtime通常比libtorch更加轻量，启动速度更快；而在开发调试阶段，TorchScript则提供了更好的PyTorch生态兼容性，便于快速迭代测试。

```
// 使用示例
auto model = InferenceRuntime::ModelFactory::load_model("policy.pt");   // 自动识别为T
auto model = InferenceRuntime::ModelFactory::load_model("policy.onnx"); // 自动识别为O
```

3.3 观测缓冲区（Observation Buffer）

许多先进的强化学习策略需要利用历史观测信息来做出决策，这对于处理部分可观测问题和提高策略的时序一致性非常重要。观测缓冲区模块`ObservationBuffer`提供了高效的历史数据管理功能，采用循环缓冲区的数据结构来存储最近若干帧的观测数据。缓冲区支持两种优先级模式：time优先级按时间顺序排列历史观测，适用于需要时序信息的策略；term优先级按观测项分组排列，适用于特定的网络架构。通过配置文件中的`observations_history`参数，用户可以灵活指定需要使用哪些历史帧的观测数据。

```
class ObservationBuffer
{
public:
    ObservationBuffer(int num_envs,
                      const std::vector<int>& obs_dims,
                      int history_length,
                      const std::string& priority = "time");

    void reset(int env_idx);                                  // 重置指定环境的缓冲区
    void insert(const std::vector<float>& obs);               // 插入新观测
    std::vector<float> get_obs_vec(const std::vector<int>& indices);  // 获取指定历史帧
};
```

3.4 循环控制器（Loop Control）

机器人控制需要严格的实时性保证，循环控制器模块`LoopFunc`提供了精确的周期性任务执行能力。每个循环控制器可以指定执行周期、回调函数以及可选的CPU亲和性设置。在实际部署中，框架通常会启动多个并行的控制循环：键盘输入循环以较低频率（如20Hz）运行处理用户输入；控制循环以较高频率（如200Hz）运行以保证控制精度；神经网络推理循环则以中等频率（如50Hz）运行以平衡计算负担和响应速度。这种多循环设计采用生产者-消费者模式，推理循环将计算结果推送到并发队列，控制循环从队列中取出最新的控制命令，既保证了控制的实时性，又避免了神经网络推理成为性能瓶颈。

```
class LoopFunc
{
public:
    LoopFunc(const std::string& name,
             float period_ms,                    // 执行周期（毫秒）
             std::function<void()> callback,     // 回调函数
             int cpu_affinity = -1);             // CPU亲和性（可选）

    void start();     // 启动循环
    void shutdown();  // 停止循环
};
```

在实际部署中，框架通常会启动多个并行的控制循环：

```
// D1实机部署示例
this->loop_keyboard = std::make_shared<LoopFunc>("loop_keyboard", 0.05,
    std::bind(&RL_Real::KeyboardInterface, this));
this->loop_control = std::make_shared<LoopFunc>("loop_control", 0.005,  // 200Hz
    std::bind(&RL_Real::RobotControl, this));
this->loop_rl = std::make_shared<LoopFunc>("loop_rl", 0.02,             // 50Hz
    std::bind(&RL_Real::RunModel, this));
```

4. 安装与配置

本节将详细介绍rl_sar框架的安装配置流程，包括依赖安装、编译选项以及配置文件的说明。

4.1 环境要求

rl_sar框架支持以下操作系统和ROS版本：

操作系统	ROS版本	状态
Ubuntu 20.04	ROS Noetic / ROS2 Foxy	完全支持
Ubuntu 22.04	ROS2 Humble	完全支持
macOS	无ROS（仅MuJoCo仿真）	实验性支持

4.2 获取源代码

首先从GitHub克隆仓库，使用`--recursive`选项以同时获取所有子模块：

```
git clone --recursive --depth 1 https://github.com/fan-ziqi/rl_sar.git
```

如需更新已有仓库：

```
git pull
git submodule update --init --recursive --recommend-shallow --progress
```

4.3 安装依赖

4.3.1 基础依赖

根据操作系统安装必要的开发库：

```
# Ubuntu系统
sudo apt install cmake g++ build-essential libyaml-cpp-dev libeigen3-dev \
    libboost-all-dev libspdlog-dev libfmt-dev libtbb-dev liblcm-dev

# macOS系统
brew install boost lcm yaml-cpp tbb libomp pkg-config glfw
```

4.3.2 ROS依赖（可选）

如果需要使用Gazebo仿真，需要安装ROS相关依赖：

```
# ROS Noetic (Ubuntu 20.04)
sudo apt install ros-noetic-teleop-twist-keyboard ros-noetic-controller-interface \
    ros-noetic-gazebo-ros-control ros-noetic-joint-state-controller \
    ros-noetic-effort-controllers ros-noetic-joint-trajectory-controller \
    ros-noetic-joy ros-noetic-ros-control ros-noetic-ros-controllers \
```

```
    ros-noetic-controller-manager

# ROS2 Foxy/Humble
sudo apt install ros-$ROS_DISTRO-teleop-twist-keyboard ros-$ROS_DISTRO-ros2-control \
    ros-$ROS_DISTRO-ros2-controllers ros-$ROS_DISTRO-control-toolbox \
    ros-$ROS_DISTRO-robot-state-publisher ros-$ROS_DISTRO-joint-state-publisher-gui \
    ros-$ROS_DISTRO-gazebo-ros2-control ros-$ROS_DISTRO-gazebo-ros-pkgs \
    ros-$ROS_DISTRO-xacro
```

4.4 编译项目

rl_sar提供了统一的编译脚本`build.sh`，支持多种编译模式：

```
# 编译所有ROS包（默认模式）
./build.sh

# 仅编译指定的包
./build.sh package1 package2
```

编译完成后，可执行文件位于以下位置：

- ROS模式：`devel/lib/rl_sar/`（ROS1）或`install/rl_sar/lib/rl_sar/`（ROS2）
- CMake模式：`cmake_build/bin/`

```
(base) lenovo@lenovo-ThinkStation-P5:/media/bigdisk/rl_sar$ ./build.sh
--------------------------------------------------------------
[Setting up Inference Runtime]
[INFO] Checking inference libraries...
[INFO] Checking inference runtimes...
[SUCCESS] LibTorch already exists and is valid
[SUCCESS] ONNX Runtime already exists and is valid
[SUCCESS] All inference runtimes are ready
[SUCCESS] Inference runtime setup completed!
--------------------------------------------------------------
[Setting up Robot Descriptions]
[INFO] Checking robot description files...
--------------------------------------------------------------
[Robot Descriptions Setup]
[SUCCESS] Robot descriptions are up-to-date (version: 1.0.1)
[INFO] Installation path: /media/bigdisk/rl_sar/src/rl_sar_zoo
[SUCCESS] Robot descriptions setup completed!
--------------------------------------------------------------
[Running ROS Build]
--------------------------------------------------------------
[Cleaning Existing Symlinks]
[INFO] Removing all existing package.xml symlinks...
[SUCCESS] Removed all existing symlinks
--------------------------------------------------------------
[Checking for Incompatible Build Artifacts]
[SUCCESS] No incompatible build artifacts found
--------------------------------------------------------------
[Creating Symlinks for All Packages]
[SUCCESS] Created symlinks for: rl_sar robot_joint_controller go2_description b2w_description gr1t1_description g1_description l4w4_description b2_description gr1
2_description lite3_description tita_description a1_description go2w_description robot_msgs
--------------------------------------------------------------
[Using colcon build]
[INFO] Building all packages...
Starting >>> robot_msgs
Starting >>> a1_description
Starting >>> b2_description
Starting >>> b2w_description
Starting >>> g1_description
Starting >>> go2_description
Starting >>> go2w_description
Starting >>> gr1t1_description
Starting >>> gr1t2_description
Starting >>> l4w4_description
Starting >>> lite3_description
Starting >>> tita_description
Finished <<< go2_description [0.51s]
Finished <<< b2_description [0.60s]
Finished <<< a1_description [0.66s]
```

下面是其他编译的方式比如我们要使用mojoco这些

```
```

```
# 使用CMake编译（不依赖ROS，用于实机部署）
./build.sh -m  # 或 ./build.sh --cmake

# 编译并启用MuJoCo仿真支持
./build.sh -mj  # 或 ./build.sh --mujoco

# 清理编译产物
./build.sh -c  # 或 ./build.sh --clean
```

```
[ 28%] Linking CXX static library lib/librl_sdk.a
[ 28%] Built target rl_sdk
[ 32%] Building CXX object CMakeFiles/rl_real_go2.dir/src/rl_real_go2.cpp.o
[ 35%] Building CXX object CMakeFiles/rl_real_lite3.dir/library/thirdparty/robot_sdk/deeprobotics/gamepad/src/gamepad.cpp.o
[ 39%] Building CXX object CMakeFiles/rl_real_lite3.dir/src/rl_real_lite3.cpp.o
[ 42%] Building CXX object CMakeFiles/rl_real_lite3.dir/library/thirdparty/robot_sdk/deeprobotics/gamepad/src/retroid_gamepad.cpp.o
[ 46%] Building CXX object CMakeFiles/rl_real_lite3.dir/library/thirdparty/robot_sdk/deeprobotics/gamepad/src/udp_receiver.cpp.o
[ 53%] Building CXX object CMakeFiles/rl_real_a1.dir/src/rl_real_a1.cpp.o
[ 53%] Building CXX object CMakeFiles/rl_real_g1.dir/src/rl_real_g1.cpp.o
[ 57%] Building CXX object CMakeFiles/rl_sim_mujoco.dir/src/rl_sim_mujoco.cpp.o
[ 60%] Building CXX object CMakeFiles/rl_sim_mujoco.dir/library/thirdparty/mujoco_simulate/platform_ui_adapter.cc.o
[ 67%] Building CXX object CMakeFiles/rl_sim_mujoco.dir/library/thirdparty/mujoco_simulate/glfw_dispatch.cc.o
[ 67%] Building CXX object CMakeFiles/rl_sim_mujoco.dir/library/thirdparty/mujoco_simulate/glfw_adapter.cc.o
[ 71%] Building CXX object CMakeFiles/rl_real_l4w4.dir/src/rl_real_l4w4.cpp.o
[ 75%] Building CXX object CMakeFiles/rl_sim_mujoco.dir/library/thirdparty/mujoco_simulate/simulate.cc.o
[ 78%] Building CXX object CMakeFiles/rl_sim_mujoco.dir/library/thirdparty/joystick/joystick.cc.o
[ 82%] Linking CXX executable bin/rl_real_lite3
[ 85%] Linking CXX executable bin/rl_real_l4w4
[ 89%] Linking CXX executable bin/rl_real_a1
[ 89%] Built target rl_real_lite3
[ 89%] Built target rl_real_l4w4
[ 89%] Built target rl_real_a1
[ 92%] Linking CXX executable bin/rl_sim_mujoco
[ 92%] Built target rl_sim_mujoco
[ 96%] Linking CXX executable bin/rl_real_g1
[100%] Linking CXX executable bin/rl_real_go2
[100%] Built target rl_real_g1
[100%] Built target rl_real_go2
[SUCCESS] MuJoCo build completed!
(base) lenovo@lenovo-ThinkStation-P5:/media/bigdisk/rl_sar$ ./cmake_build/bin/rl_sim_mujoco g1 scene_29dof
[INFO]    [FSMManager] Registered type: a1
[INFO]    [FSMManager] Registered type: b2
[INFO]    [FSMManager] Registered type: b2w
[INFO]    [FSMManager] Registered type: g1
[INFO]    [FSMManager] Registered type: go2
[INFO]    [FSMManager] Registered type: go2w
[INFO]    [FSMManager] Registered type: gr1t1
[INFO]    [FSMManager] Registered type: gr1t2
[INFO]    [FSMManager] Registered type: l4w4
[INFO]    [FSMManager] Registered type: lite3
[INFO]    [FSMManager] Registered type: tita
[INFO]    [MuJoCo] Launching...
[INFO]    [MuJoCo] Version: 3.2.7
[INFO]    [MuJoCo] Data prepared
[ERROR]   Joystick [/dev/input/js0] open failed.
[NOTE]    Entered passive mode. Press '0' (Keyboard) or 'A' (Gamepad) to switch to RLFSMStateGetUp.
[INFO]    [FSM] Set initial state: RLFSMStatePassive
[INFO]    [FSMManager] FSM created for type: g1
[INFO]    [Loop] Loop start - name: loop_control, period: 5ms, cpu: unspecified
[INFO]    [Loop] Loop start - name: loop_rl, period: 20ms, cpu: unspecified
[INFO]    [Loop] Loop start - name: loop_keyboard, period: 50ms, cpu: unspecified
[INFO]    [Loop] Loop start - name: loop_joystick, period: 10ms, cpu: unspecified
[INFO]    RL_Sim start
```

4.5 配置文件详解

rl_sar使用YAML格式的配置文件来管理机器人参数和策略配置。配置文件位于`policy/`目录下，采用两级结构：

```
policy/
├── <ROBOT>/
│   ├── base.yaml              # 机器人基础物理参数
│   └── <CONFIG>/
│       ├── config.yaml        # 策略特定配置
│       └── <MODEL>.pt         # 训练好的模型文件
```

4.5.1 base.yaml - 机器人基础配置

`base.yaml`定义了机器人的物理参数，这些参数通常与具体的策略无关：

```yaml
D1:
  dt: 0.005                    # 控制周期（秒）
  decimation: 4                # 策略执行频率 = 1/(dt * decimation)
  num_of_dofs: 12              # 自由度数量

  # PD控制增益（用于位置插值过渡）
  fixed_kp: [80.0, 80.0, 80.0, 80.0, 80.0, 80.0,
             80.0, 80.0, 80.0, 80.0, 80.0, 80.0]
  fixed_kd: [3.0, 3.0, 3.0, 3.0, 3.0, 3.0,
             3.0, 3.0, 3.0, 3.0, 3.0, 3.0]

  # 关节力矩限制
  torque_limits: [23.5, 23.5, 23.5, 23.5, 23.5, 23.5,
                  23.5, 23.5, 23.5, 23.5, 23.5, 23.5]

  # 默认站立姿态（关节角度，弧度）
  default_dof_pos: [0.00, 0.80, -1.50, 0.00, 0.80, -1.50,
                    0.00, 0.80, -1.50, 0.00, 0.80, -1.50]

  # 关节名称（与URDF对应）
  joint_names: ["FR_hip_joint", "FR_thigh_joint", "FR_calf_joint",
                "FL_hip_joint", "FL_thigh_joint", "FL_calf_joint",
                "RR_hip_joint", "RR_thigh_joint", "RR_calf_joint",
                "RL_hip_joint", "RL_thigh_joint", "RL_calf_joint"]

  # 关节映射（SDK顺序到训练顺序的映射）
  joint_mapping: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
```

4.5.2 config.yaml - 策略配置

`config.yaml`定义了特定策略的参数，这些参数需要与训练时的设置保持一致：

```yaml
D1/himloco:
  model_name: "himloco.pt"      # 模型文件名
  num_observations: 45          # 观测向量维度

  # 观测组成（顺序很重要！）
  observations: ["commands", "ang_vel", "gravity_vec",
                 "dof_pos", "dof_vel", "actions"]

  # 历史观测配置
  observations_history: [0, 1, 2, 3, 4, 5]  # 使用最近6帧
  observations_history_priority: "time"       # 时间优先级

  # 观测和动作裁剪
  clip_obs: 100.0
  clip_actions_lower: [-100, -100, -100, -100, -100, -100,
                       -100, -100, -100, -100, -100, -100]
  clip_actions_upper: [100, 100, 100, 100, 100, 100,
                       100, 100, 100, 100, 100, 100]

  # RL控制器PD增益
  rl_kp: [40, 40, 40, 40, 40, 40, 40, 40, 40, 40, 40, 40]
  rl_kd: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

  # 缩放因子（必须与训练时一致）
```

```
  action_scale: [0.125, 0.25, 0.25, 0.125, 0.25, 0.25,
                 0.125, 0.25, 0.25, 0.125, 0.25, 0.25]
  lin_vel_scale: 2.0
  ang_vel_scale: 0.25
  dof_pos_scale: 1.0
  dof_vel_scale: 0.05
  commands_scale: [2.0, 2.0, 0.25]

  # 策略特定的默认姿态和关节映射
  default_dof_pos: [0.1, 0.8, -1.5, -0.1, 0.8, -1.5,
                    0.1, 1.0, -1.5, -0.1, 1.0, -1.5]
  joint_mapping: [3, 4, 5, 0, 1, 2, 9, 10, 11, 6, 7, 8]
```

需要特别注意的是`joint_mapping`参数，它定义了机器人SDK中的关节顺序与训练环境中关节顺序之间的映射关系。如果这个映射配置错误，会导致机器人产生完全错误的运动行为，甚至可能造成硬件损坏。

5. 控制循环与核心算法实现

本节将深入分析rl_sar框架中控制循环的实现细节，包括观测计算、模型推理和输出计算等核心算法。

5.1 控制循环数据流

下图展示了强化学习控制循环中的数据流向：



整个控制循环可以分为以下几个关键步骤：

1. **GetState()**: 从机器人硬件或仿真器获取当前状态
2. **ComputeObservation()**: 将原始状态转换为策略网络的输入

3. **Forward()**: 执行神经网络推理，输出动作
4. **ComputeOutput()**: 将动作转换为关节控制命令
5. **SetCommand()**: 将控制命令发送给机器人

5.2 观测计算详解

`ComputeObservation()`函数是连接机器人状态和策略网络的桥梁。它根据配置文件中定义的观测类型，从机器人状态中提取相应的数据，并进行必要的坐标变换和缩放处理：

```
std::vector<float> RL::ComputeObservation()
{
    std::vector<std::vector<float>> obs_list;

    // 遍历配置文件中定义的观测类型
    for (const std::string &observation : this->params.Get<std::vector<std::string>>
    {
        if (observation == "lin_vel")
        {
            // 线速度观测（通常需要状态估计器）
            obs_list.push_back(this->obs.lin_vel * this->params.Get<float>("lin_vel_
        }
        else if (observation == "ang_vel")
        {
            // 角速度观测 - 需要注意坐标系
            // 在ROS1 Gazebo中角速度是世界坐标系，需要转换到机体坐标系
            // 在ROS2 Gazebo、MuJoCo和真实机器人中，角速度已经是机体坐标系
            if (this->ang_vel_axis == "body")
            {
                obs_list.push_back(this->obs.ang_vel * this->params.Get<float>("ang_
            }
            else if (this->ang_vel_axis == "world")
            {
                // 使用四元数将世界坐标系的角速度转换到机体坐标系
                obs_list.push_back(QuatRotateInverse(this->obs.base_quat, this->obs.
                                   * this->params.Get<float>("ang_vel_scale"));
            }
        }
        else if (observation == "gravity_vec")
        {
            // 重力向量 - 用于感知机器人姿态
            // 将世界坐标系的重力向量[0,0,-1]转换到机体坐标系
            obs_list.push_back(QuatRotateInverse(this->obs.base_quat, this->obs.grav
        }
        else if (observation == "commands")
        {
            // 用户命令（前进速度、侧移速度、转向速度）
            obs_list.push_back(this->obs.commands * this->params.Get<std::vector<flo
        }
        else if (observation == "dof_pos")
        {
            // 关节位置 - 相对于默认姿态的偏差
            std::vector<float> dof_pos_rel = this->obs.dof_pos
                                           - this->params.Get<std::vector<float>>("d
```

```
            // 对于轮式机器人，轮子的位置观测设为0
            for (int i : this->params.Get<std::vector<int>>("wheel_indices"))
            {
                dof_pos_rel[i] = 0.0f;
            }
            obs_list.push_back(dof_pos_rel * this->params.Get<float>("dof_pos_scale")
        }
        else if (observation == "dof_vel")
        {
            // 关节速度
            obs_list.push_back(this->obs.dof_vel * this->params.Get<float>("dof_vel_s
        }
        else if (observation == "actions")
        {
            // 上一时刻的动作（用于时序一致性）
            obs_list.push_back(this->obs.actions);
        }
    }

    // 将所有观测拼接成一个向量
    std::vector<float> obs;
    for (const auto& obs_vec : obs_list)
    {
        obs.insert(obs.end(), obs_vec.begin(), obs_vec.end());
    }

    // 裁剪观测值到合理范围，防止异常值影响策略
    std::vector<float> clamped_obs = clamp(obs,
        -this->params.Get<float>("clip_obs"),
        this->params.Get<float>("clip_obs"));

    return clamped_obs;
}
```

这段代码中有几个关键点值得注意：

坐标系转换：不同的仿真器和真实机器人可能使用不同的坐标系约定。例如，角速度可能在世界坐标系或机体坐标系中表示。`QuatRotateInverse`函数用于将世界坐标系中的向量转换到机体坐标系，这对于保证Sim-to-Real的一致性至关重要。

相对位置表示：关节位置使用相对于默认姿态的偏差来表示，而不是绝对角度。这种表示方式使得策略更容易学习，因为输出的动作可以直接解释为相对于默认姿态的调整量。

缩放因子：所有观测都乘以相应的缩放因子，这些因子必须与训练时使用的值完全一致。缩放的目的是将不同量纲的物理量归一化到相近的数值范围，有助于神经网络的训练和推理。

5.3 模型推理实现

`Forward()`函数负责执行神经网络推理。以D1实机部署为例：

```cpp
std::vector<float> RL_Real::Forward()
{
    // 尝试获取模型锁，避免在模型重新初始化时阻塞
    std::unique_lock<std::mutex> lock(this->model_mutex, std::try_to_lock);
    if (!lock.owns_lock())
    {
        // 如果模型正在重新初始化，返回上一次的动作
        std::cout << LOGGER::WARNING << "Model is being reinitialized, using previous
        return this->obs.actions;
    }

    // 计算当前观测
    std::vector<float> clamped_obs = this->ComputeObservation();

    std::vector<float> actions;

    // 检查是否需要使用历史观测
    if (!this->params.Get<std::vector<int>>("observations_history").empty())
    {
        // 将当前观测插入历史缓冲区
        this->history_obs_buf.insert(clamped_obs);
        // 获取指定历史帧的观测
        this->history_obs = this->history_obs_buf.get_obs_vec(
            this->params.Get<std::vector<int>>("observations_history"));
        // 使用历史观测进行推理
        actions = this->model->forward({this->history_obs});
    }
    else
    {
        // 直接使用当前观测进行推理
        actions = this->model->forward({clamped_obs});
    }

    // 裁剪动作到合理范围
    if (!this->params.Get<std::vector<float>>("clip_actions_upper").empty())
    {
        return clamp(actions,
            this->params.Get<std::vector<float>>("clip_actions_lower"),
            this->params.Get<std::vector<float>>("clip_actions_upper"));
    }
    return actions;
}
```

5.4 输出计算与PD控制

`ComputeOutput()` 函数将神经网络输出的动作转换为实际的关节控制命令。rl_sar采用位置控制模式，通过PD控制器计算所需的力矩：

```cpp
void RL::ComputeOutput(const std::vector<float> &actions,
                       std::vector<float> &output_dof_pos,
                       std::vector<float> &output_dof_vel,
                       std::vector<float> &output_dof_tau)
{
```

```
    // 1. 动作缩放
    std::vector<float> actions_scaled = actions * this->params.Get<std::vector<float
    // 2. 分离位置动作和速度动作（用于轮式机器人）
    std::vector<float> pos_actions_scaled = actions_scaled;
    std::vector<float> vel_actions_scaled(actions.size(), 0.0f);
    // 对于轮子关节，使用速度控制而非位置控制
    for (int i : this->params.Get<std::vector<int>>("wheel_indices"))
    {
        pos_actions_scaled[i] = 0.0f;
        vel_actions_scaled[i] = actions_scaled[i];
    }
    // 3. 计算目标位置（动作 + 默认姿态）
    output_dof_pos = pos_actions_scaled + this->params.Get<std::vector<float>>("defau
    // 4. 计算目标速度
    output_dof_vel = vel_actions_scaled;
    // 5. 计算PD控制力矩
    // tau = kp * (pos_target - pos_current) - kd * vel_current
    std::vector<float> all_actions_scaled = pos_actions_scaled + vel_actions_scaled;
    output_dof_tau = this->params.Get<std::vector<float>>("rl_kp")
                   * (all_actions_scaled + this->params.Get<std::vector<float>>("defa
                      - this->obs.dof_pos)
                   - this->params.Get<std::vector<float>>("rl_kd") * this->obs.dof_ve
    // 6. 力矩安全限制
    output_dof_tau = clamp(output_dof_tau,
        -this->params.Get<std::vector<float>>("torque_limits"),
        this->params.Get<std::vector<float>>("torque_limits"));
}
```

这个函数实现了标准的PD位置控制器，其中：

- **kp（位置增益）**：决定了位置误差对力矩的贡献
- **kd（速度增益）**：提供阻尼效果，防止振荡

力矩限制是一个重要的安全机制，防止异常的控制命令损坏电机或机械结构。

5.5 实机控制主循环

以D1机器人为例，实机控制的主循环实现如下：

```
void RL_Real::RobotControl()
{
    // 1. 获取机器人当前状态
    this->GetState(&this->robot_state);
    // 2. 运行状态控制器（FSM）
    this->StateController(&this->robot_state, &this->robot_command);
```

```
    // 3. 清除输入状态（防止按键重复触发）
    this->control.ClearInput();

    // 4. 发送控制命令到机器人
    this->SetCommand(&this->robot_command);
}

void RL_Real::RunModel()
{
    // 仅在RL控制状态下运行模型推理
    if (this->rl_init_done)
    {
        this->episode_length_buf += 1;

        // 更新观测数据
        this->obs.ang_vel = this->robot_state.imu.gyroscope;
        this->obs.commands = {this->control.x, this->control.y, this->control.yaw};
        this->obs.base_quat = this->robot_state.imu.quaternion;
        this->obs.dof_pos = this->robot_state.motor_state.q;
        this->obs.dof_vel = this->robot_state.motor_state.dq;

        // 执行模型推理
        this->obs.actions = this->Forward();

        // 计算控制输出
        this->ComputeOutput(this->obs.actions, this->output_dof_pos,
                            this->output_dof_vel, this->output_dof_tau);

        // 将输出推送到队列，供控制循环使用
        if (!this->output_dof_pos.empty())
            output_dof_pos_queue.push(this->output_dof_pos);
        if (!this->output_dof_vel.empty())
            output_dof_vel_queue.push(this->output_dof_vel);
    }
}
```

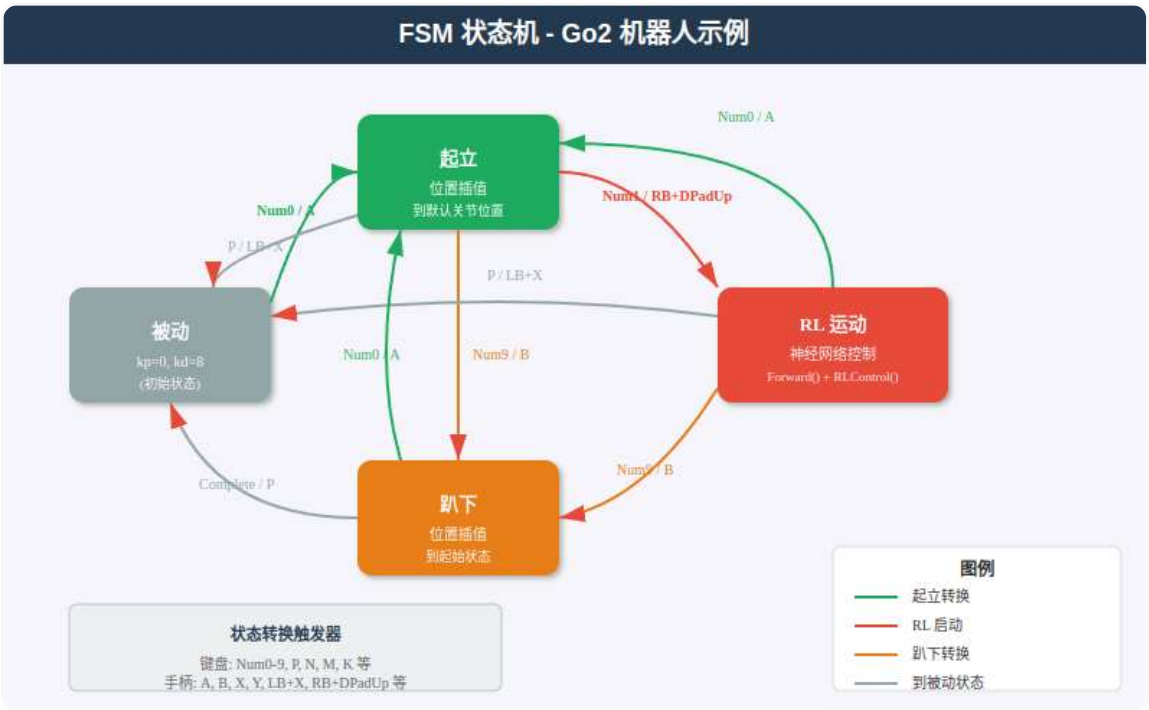
这里使用了生产者-消费者模式：`RunModel()`以较低频率（如50Hz）运行，将计算结果推送到并发队列；`RobotControl()`以较高频率（如200Hz）运行，从队列中取出最新的控制命令发送给机器人。这种设计既保证了控制的实时性，又避免了神经网络推理成为性能瓶颈。

6. 有限状态机（FSM）设计

机器人控制系统需要管理多种运行状态，如待机、起身、行走、趴下等。rl_sar框架采用有限状态机（Finite State Machine, FSM）来管理这些状态之间的转换，确保机器人在不同状态下执行正确的控制逻辑。

6.1 FSM架构设计

下图展示了D1机器人的状态机设计：



框架的FSM采用了经典的状态模式设计，主要包含以下组件：

```cpp
// 状态基类
class FSMState
{
public:
    FSMState(std::string name) : state_name_(std::move(name)) {}
    virtual ~FSMState() = default;

    virtual void Enter()= 0;            // 进入状态时执行
    virtual void Run()= 0;              // 状态运行时每帧执行
    virtual void Exit()= 0;             // 退出状态时执行
    virtual std::string CheckChange();  // 检查是否需要状态转换

    const std::string &GetStateName() const{ return state_name_; }

protected:
    std::string state_name_;
};

// 状态机管理器
class FSM
{
public:
    void AddState(std::shared_ptr<FSMState> state);
    void SetInitialState(const std::string &name);
    void RequestStateChange(const std::string& state_name);
    void Run();

private:
    std::unordered_map<std::string, std::shared_ptr<FSMState>> states_;
```

```
    std::shared_ptr<FSMState> current_state_;
    std::shared_ptr<FSMState> next_state_;
    Mode mode_;  // NORMAL or CHANGE
};
```

6.2 RL专用状态基类

为了简化强化学习控制状态的实现，框架提供了`RLFSMState`基类，封装了常用的功能：

```
class RLFSMState : public FSMState
{
public:
    RLFSMState(RL& rl, const std::string& name)
        : FSMState(name), rl(rl), fsm_state(nullptr), fsm_command(nullptr) {}

    RL& rl;  // 对RL控制器的引用
    const RobotState<float> *fsm_state;    // 当前机器人状态
    RobotCommand<float> *fsm_command;      // 输出控制命令

    // 位置插值函数 - 用于平滑过渡
    bool Interpolate(float& percent,
                     const std::vector<float>& start_pos,
                     const std::vector<float>& target_pos,
                     float duration_seconds,
                     const std::string& description = "",
                     bool use_fixed_gains = true);

    // RL控制函数 - 从队列获取控制命令
    void RLControl();
};
```

`Interpolate()`函数实现了关节位置的平滑插值，用于起身和趴下等过渡动作：

```
bool RLFSMState::Interpolate(float& percent,
                             const std::vector<float>& start_pos,
                             const std::vector<float>& target_pos,
                             float duration_seconds,
                             const std::string& description,
                             bool use_fixed_gains)
{
    if (percent >= 1.0f) return false;

    // 计算插值步长
    int required_frames = std::max(1,
        static_cast<int>(std::ceil(duration_seconds / rl.params.Get<float>("dt"))));
    float step = 1.0f / required_frames;

    percent += step;
    percent = std::min(percent, 1.0f);

    // 选择PD增益
```

```
    auto kp = use_fixed_gains ? rl.params.Get<std::vector<float>>("fixed_kp")
                              : rl.params.Get<std::vector<float>>("rl_kp");
    auto kd = use_fixed_gains ? rl.params.Get<std::vector<float>>("fixed_kd")
                              : rl.params.Get<std::vector<float>>("rl_kd");

    // 线性插值计算目标位置
    for (int i = 0; i < rl.params.Get<int>("num_of_dofs"); ++i)
    {
        fsm_command->motor_command.q[i] = (1 - percent) * start_pos[i]
                                          + percent * target_pos[i];
        fsm_command->motor_command.dq[i] = 0;
        fsm_command->motor_command.kp[i] = kp[i];
        fsm_command->motor_command.kd[i] = kd[i];
        fsm_command->motor_command.tau[i] = 0;
    }

    // 显示进度
    if (!description.empty())
    {
        LOGGER::PrintProgress(percent, description);
    }

    return percent < 1.0f;
}
```

6.3 D1机器人状态实现

以D1机器人为例，框架定义了以下状态：

6.3.1 Passive状态（被动模式）

```
class RLFSMStatePassive : public RLFSMState
{
public:
    RLFSMStatePassive(RL *rl) : RLFSMState(*rl, "RLFSMStatePassive") {}

    void Enter() override
    {
        std::cout << LOGGER::NOTE << "Entered passive mode. "
                  << "Press '0' or 'A' to switch to GetUp." << std::endl;
    }

    void Run() override
    {
        // 被动模式：kp=0, kd=8，机器人可以被自由移动
        for (int i = 0; i < rl.params.Get<int>("num_of_dofs"); ++i)
        {
            fsm_command->motor_command.dq[i] = 0;
            fsm_command->motor_command.kp[i] = 0;   // 无位置控制
            fsm_command->motor_command.kd[i] = 8;   // 仅阻尼
            fsm_command->motor_command.tau[i] = 0;
        }
    }

    void Exit() override{}
```

```
    std::string CheckChange() override
    {
        // 按下Num0或手柄A键切换到起身状态
        if (rl.control.current_keyboard == Input::Keyboard::Num0 ||
            rl.control.current_gamepad == Input::Gamepad::A)
        {
            return "RLFSMStateGetUp";
        }
        return state_name_;
    }
};
```

6.3.2 GetUp状态（起身）

```
class RLFSMStateGetUp : public RLFSMState
{
public:
    float percent_pre_getup = 0.0f;
    float percent_getup = 0.0f;
    // 预备姿态（蹲伏姿态）
    std::vector<float> pre_running_pos = {
        0.00, 1.36, -2.65,  // FR
        0.00, 1.36, -2.65,  // FL
        0.00, 1.36, -2.65,  // RR
        0.00, 1.36, -2.65   // RL
    };

    void Enter() override
    {
        percent_pre_getup = 0.0f;
        percent_getup = 0.0f;
        rl.now_state = *fsm_state;
        rl.start_state = rl.now_state;
    }

    void Run() override
    {
        // 两阶段起身：先到预备姿态，再到站立姿态
        if (Interpolate(percent_pre_getup, rl.now_state.motor_state.q,
                        pre_running_pos, 1.0f, "Pre Getting up", true))
            return;
        if (Interpolate(percent_getup, pre_running_pos,
                        rl.params.Get<std::vector<float>>("default_dof_pos"),
                        2.0f, "Getting up", true))
            return;
    }

    std::string CheckChange() override
    {
        if (rl.control.current_keyboard == Input::Keyboard::P)
            return "RLFSMStatePassive";
        if (percent_getup >= 1.0f)
        {
            if (rl.control.current_keyboard == Input::Keyboard::Num1)
                return "RLFSMStateRLLocomotion";
            if (rl.control.current_keyboard == Input::Keyboard::Num9)
                return "RLFSMStateGetDown";
        }
```

```
        return state_name_;
    }
};
```

6.3.3 RLLocomotion状态（强化学习行走）



```cpp
class RLFSMStateRLLocomotion : public RLFSMState
{
public:
    void Enter() override
    {
        rl.episode_length_buf = 0;

        // 加载策略配置和模型
        rl.config_name = "himloco";
        std::string robot_config_path = rl.robot_name + "/" + rl.config_name;
        try
        {
            rl.InitRL(robot_config_path);
            rl.now_state = *fsm_state;
        }
        catch (const std::exception& e)
        {
            std::cout << LOGGER::ERROR << "InitRL() failed: " << e.what() << std::end
            rl.rl_init_done = false;
            rl.fsm.RequestStateChange("RLFSMStatePassive");
        }
    }

    void Run() override
    {
        if (!rl.rl_init_done)
            rl.rl_init_done = true;

        // 显示当前控制命令
        std::cout << "\r\033[K" << std::flush << LOGGER::INFO
                  << "RL Controller [" << rl.config_name << "] "
                  << "x:" << rl.control.x
                  << " y:" << rl.control.y
                  << " yaw:" << rl.control.yaw << std::flush;

        // 执行RL控制
        RLControl();
    }

    void Exit() override
    {
        rl.rl_init_done = false;
    }

    std::string CheckChange() override
    {
        if (rl.control.current_keyboard == Input::Keyboard::P)
            return "RLFSMStatePassive";
        if (rl.control.current_keyboard == Input::Keyboard::Num9)
            return "RLFSMStateGetDown";
        return state_name_;
    }
};
```

6.4 FSM工厂模式

为了支持不同机器人的状态机配置，框架采用了工厂模式和自动注册机制：

```cpp
class D1FSMFactory : public FSMFactory
{
public:
    D1FSMFactory(const std::string& initial) : initial_state_(initial) {}

    std::shared_ptr<FSMState> CreateState(void *context, const std::string &state_nar
    {
        RL *rl = static_cast<RL *>(context);
        if (state_name == "RLFSMStatePassive")
            return std::make_shared<d1_fsm::RLFSMStatePassive>(rl);
        else if (state_name == "RLFSMStateGetUp")
            return std::make_shared<d1_fsm::RLFSMStateGetUp>(rl);
        else if (state_name == "RLFSMStateGetDown")
            return std::make_shared<d1_fsm::RLFSMStateGetDown>(rl);
        else if (state_name == "RLFSMStateRLLocomotion")
            return std::make_shared<d1_fsm::RLFSMStateRLLocomotion>(rl);
        return nullptr;
    }

    std::string GetType() const override{ return "d1"; }

    std::vector<std::string> GetSupportedStates() const override
    {
        return {"RLFSMStatePassive", "RLFSMStateGetUp",
                "RLFSMStateGetDown", "RLFSMStateRLLocomotion"};
    }

    std::string GetInitialState() const override{ return initial_state_; }

private:
    std::string initial_state_;
};

// 自动注册宏
REGISTER_FSM_FACTORY(D1FSMFactory, "RLFSMStatePassive")
```

通过这种设计，添加新机器人时只需要创建对应的FSM工厂类并使用宏注册，框架会自动识别并加载相应的状态机。

7. 二次开发指南

rl_sar框架的模块化设计使得添加新机器人或自定义功能变得相对简单。本节将以Agibot D1四足机器人为实际案例，详细介绍如何进行二次开发，包括文件结构创建、SDK集成、关节映射配置、FSM状态机实现等关键步骤，帮助开发者快速将自己的机器人平台集成到rl_sar框架中。在开始二次开发之前，建议开发者先熟悉框架的整体架构和核心模块，这将有助于更好地理解各个组件之间的关系和数据流向。

7.1 添加新机器人的完整文件结构

添加新机器人需要创建或修改多个文件，这些文件分布在不同的目录中，各自承担不同的功能职责。以D1机器人为例，完整的文件结构如下所示。开发者在添加新机器人时，建议按照这个结构组织文件，以保持与框架的一致性，便于后续维护和升级。需要特别注意的是，文件命名应该遵循框架的命名规范，机器人名称应该使用小写字母，并在所有相关文件中保持一致。

```
rl_sar/
├── src/rl_sar_zoo/<ROBOT>_description/    # 机器人描述文件包
│   ├── CMakeLists.txt                     # CMake构建配置
│   ├── package.ros1.xml                   # ROS1包描述文件
│   ├── package.ros2.xml                   # ROS2包描述文件
│   ├── urdf/robot_description.urdf        # URDF机器人模型
│   ├── xacro/robot.xacro                  # Xacro主文件（包含URDF和Gazebo配置）
│   ├── xacro/gazebo.xacro                 # Gazebo仿真专用配置
│   ├── mjcf/robot.xml                     # MuJoCo模型文件
│   ├── mjcf/scene.xml                     # MuJoCo场景文件
│   ├── mjcf/assets/                       # MuJoCo网格资源目录
│   ├── meshes/                            # STL/DAE网格文件目录
│   ├── config/robot_control.yaml          # ROS1控制器配置
│   ├── config/robot_control_ros2.yaml     # ROS2控制器配置
│   └── launch/gazebo.launch               # Gazebo启动文件
│
├── policy/<ROBOT>/                        # 策略配置目录
│   ├── base.yaml                          # 机器人基础物理参数
│   └── <CONFIG>/                          # 特定策略配置子目录
│       ├── config.yaml                    # 策略参数配置
│       └── <MODEL>.pt                     # 训练好的模型文件（.pt或.onnx）
│
├── src/rl_sar/fsm_robot/                  # FSM状态机目录
│   ├── fsm_<ROBOT>.hpp                    # 机器人专用状态机定义
│   └── fsm_all.hpp                        # FSM包含列表（需要添加新机器人的头文件）
│
├── src/rl_sar/include/                    # 头文件目录
│   └── rl_real_<ROBOT>.hpp                # 实机控制类头文件
│
└── src/rl_sar/src/                        # 源代码目录
    └── rl_real_<ROBOT>.cpp                # 实机控制类实现
```

7.2 配置文件详解与注意事项

配置文件是连接训练环境和部署环境的关键桥梁，参数的正确性直接决定了策略能否正常工作。以D1机器人为例，`base.yaml`定义了机器人的基础物理参数，这些参数通常与具体的策略无关，但必须与机器人的实际硬件规格相匹配。特别需要注意的是`joint_names`必须与URDF中定义的关节名称完全一致，`torque_limits`必须根据电机的实际额定力矩进行设置，过大的力矩限制可能导致电机过热甚至损坏。

```
```

```yaml
# policy/d1/base.yaml - D1机器人基础配置示例
d1:
  dt: 0.005                     # 控制周期5ms，对应200Hz控制频率
  decimation: 4                 # 策略执行频率 = 200Hz / 4 = 50Hz
  num_of_dofs: 12               # 四足机器人12个自由度（每条腿3个关节）

  # 固定PD增益 - 用于起身/趴下等过渡动作
  # 这些增益通常设置得较高，以确保过渡动作的稳定性和精确性
  fixed_kp: [80.0, 80.0, 80.0, 80.0, 80.0, 80.0,
             80.0, 80.0, 80.0, 80.0, 80.0, 80.0]
  fixed_kd: [3.0, 3.0, 3.0, 3.0, 3.0, 3.0,
             3.0, 3.0, 3.0, 3.0, 3.0, 3.0]

  # 关节力矩限制 - 必须根据电机规格设置，D1使用33.5Nm电机
  torque_limits: [33.5, 33.5, 33.5, 33.5, 33.5, 33.5,
                  33.5, 33.5, 33.5, 33.5, 33.5, 33.5]

  # 默认站立姿态（弧度） - 机器人起身后的目标姿态
  # 顺序：FR(前右腿), FL(前左腿), RR(后右腿), RL(后左腿)
  default_dof_pos: [0.00, 0.80, -1.50, 0.00, 0.80, -1.50,
                    0.00, 0.80, -1.50, 0.00, 0.80, -1.50]

  # 关节名称 - 必须与URDF中的关节名称完全一致
  joint_names: ["FR_ABAD_JOINT", "FR_HIP_JOINT", "FR_KNEE_JOINT",
                "FL_ABAD_JOINT", "FL_HIP_JOINT", "FL_KNEE_JOINT",
                "RR_ABAD_JOINT", "RR_HIP_JOINT", "RR_KNEE_JOINT",
                "RL_ABAD_JOINT", "RL_HIP_JOINT", "RL_KNEE_JOINT"]

  # 关节映射 - 当SDK顺序与训练顺序一致时为恒等映射
  joint_mapping: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

  # 轮子索引 - 四足机器人为空，轮足机器人需要指定轮子关节索引
  wheel_indices: []
```

策略配置文件`config.yaml`定义了特定策略的参数，这些参数必须与训练时使用的配置完全一致，否则会导致策略行为异常。观测向量的组成顺序尤为重要，不同的顺序会导致神经网络接收到错误的输入，从而产生不可预测的输出。缩放因子的设置也需要特别注意，它们用于将不同量纲的物理量归一化到相近的数值范围，必须与训练时保持一致。

```yaml
# policy/d1/robot_lab/config.yaml - D1策略配置示例
d1/robot_lab:
  model_name: "policy.pt"       # 模型文件名（支持.pt和.onnx格式）
  num_observations: 45          # 观测向量总维度

  # 观测组成 - 顺序必须与训练时完全一致！
  # ang_vel(3) + gravity_vec(3) + commands(3) + dof_pos(12) + dof_vel(12) + actions(1
  observations: ["ang_vel", "gravity_vec", "commands",
                 "dof_pos", "dof_vel", "actions"]

  # 历史观测配置 - 空列表表示不使用历史观测
  observations_history: []
  observations_history_priority: "time"

  # 观测和动作裁剪范围 - 防止异常值影响策略
  clip_obs: 100.0
```

```yaml
  clip_actions_lower: [-100.0, -100.0, -100.0, -100.0, -100.0, -100.0,
                       -100.0, -100.0, -100.0, -100.0, -100.0, -100.0]
  clip_actions_upper: [100.0, 100.0, 100.0, 100.0, 100.0, 100.0,
                       100.0, 100.0, 100.0, 100.0, 100.0, 100.0]

  # RL控制器PD增益 - 通常比fixed_kp/kd小，以获得更柔顺的运动
  rl_kp: [20.0, 20.0, 20.0, 20.0, 20.0, 20.0,
          20.0, 20.0, 20.0, 20.0, 20.0, 20.0]
  rl_kd: [0.5, 0.5, 0.5, 0.5, 0.5, 0.5,
          0.5, 0.5, 0.5, 0.5, 0.5, 0.5]

  # 动作缩放因子 - 不同关节可以有不同的缩放
  # abad关节运动范围小用0.125，hip和knee关节范围大用0.25
  action_scale: [0.125, 0.25, 0.25, 0.125, 0.25, 0.25,
                 0.125, 0.25, 0.25, 0.125, 0.25, 0.25]

  # 观测缩放因子 - 必须与训练时一致
  lin_vel_scale: 2.0           # 线速度缩放
  ang_vel_scale: 0.25          # 角速度缩放
  dof_pos_scale: 1.0           # 关节位置缩放
  dof_vel_scale: 0.05          # 关节速度缩放
  commands_scale: [1.0, 1.0, 1.0]  # 命令缩放 [vx, vy, yaw_rate]
```

7.3 关节映射机制详解

关节映射是实机部署中最容易出错的环节之一，不同机器人SDK的关节顺序往往与训练环境中的顺序不同。以D1机器人为例，其SDK使用的关节顺序是按关节类型分组的（FL[0], FR[1], RL[2], RR[3]），而rl_sar框架使用的是按腿分组的顺序（FR(0-2), FL(3-5), RR(6-8), RL(9-11)）。这种差异需要在代码中进行显式的映射转换，否则会导致机器人产生完全错误的运动行为，甚至可能造成硬件损坏。

```cpp
// D1机器人的关节映射实现示例
// SDK关节顺序: FL[0], FR[1], RL[2], RR[3] 对于每种关节类型(abad, hip, knee)
// rl_sar关节顺序: FR(0-2), FL(3-5), RR(6-8), RL(9-11)
// 映射表: rl_sar腿索引 -> SDK腿索引
static const int SDK_LEG_MAP[4] = {1, 0, 3, 2}; // FR->1, FL->0, RR->3, RL->2

void RL_Real::GetState(RobotState<float> *state)
{
    // 获取SDK原始数据
    this->d1_motor_state = this->d1_lowlevel.getMotorState();

    // 获取IMU数据 - 注意四元数顺序可能需要调整
    std::vector<float> quat = this->d1_lowlevel.getQuaternion();
    std::vector<float> gyro = this->d1_lowlevel.getBodyGyro();

    // IMU数据赋值 - 确认四元数顺序为(w, x, y, z)
    if (quat.size() >= 4)
    {
        state->imu.quaternion[0] = quat[0]; // w
        state->imu.quaternion[1] = quat[1]; // x
        state->imu.quaternion[2] = quat[2]; // y
        state->imu.quaternion[3] = quat[3]; // z
    }
```

```
    if (gyro.size() >= 3)
    {
        state->imu.gyroscope[0] = gyro[0];
        state->imu.gyroscope[1] = gyro[1];
        state->imu.gyroscope[2] = gyro[2];
    }

    // 关节状态映射 - 从SDK顺序转换到rl_sar顺序
    if (this->d1_motor_state != nullptr && this->d1_lowlevel.haveMotorData())
    {
        for (int leg = 0; leg < 4; ++leg)
        {
            int sdk_leg = SDK_LEG_MAP[leg];  // 获取对应的SDK腿索引
            int base_idx = leg * 3;          // rl_sar中该腿的起始索引

            // 位置映射
            state->motor_state.q[base_idx + 0] = this->d1_motor_state->q_abad[sdk_leg
            state->motor_state.q[base_idx + 1] = this->d1_motor_state->q_hip[sdk_leg
            state->motor_state.q[base_idx + 2] = this->d1_motor_state->q_knee[sdk_leg

            // 速度映射
            state->motor_state.dq[base_idx + 0] = this->d1_motor_state->qd_abad[sdk_
            state->motor_state.dq[base_idx + 1] = this->d1_motor_state->qd_hip[sdk_le
            state->motor_state.dq[base_idx + 2] = this->d1_motor_state->qd_knee[sdk_

            // 力矩反馈映射
            state->motor_state.tau_est[base_idx + 0] = this->d1_motor_state->tau_abad
            state->motor_state.tau_est[base_idx + 1] = this->d1_motor_state->tau_hip_
            state->motor_state.tau_est[base_idx + 2] = this->d1_motor_state->tau_knee
        }
    }
}
```

控制命令的映射需要进行反向转换，将rl_sar框架计算出的控制命令转换为SDK能够识别的格式。这个过程同样需要使用映射表，确保每个关节都能收到正确的控制指令。在实际开发中，建议先在仿真环境中验证映射的正确性，再进行实机测试，以避免因映射错误导致的硬件损坏。

```
void RL_Real::SetCommand(const RobotCommand<float> *command)
{
    // 将rl_sar顺序的命令映射到SDK顺序
    for (int leg = 0; leg < 4; ++leg)
    {
        int sdk_leg = SDK_LEG_MAP[leg];  // 获取对应的SDK腿索引
        int base_idx = leg * 3;          // rl_sar中该腿的起始索引

        // 目标位置映射
        this->d1_motor_cmd.q_des_abad[sdk_leg] = command->motor_command.q[base_idx +
        this->d1_motor_cmd.q_des_hip[sdk_leg] = command->motor_command.q[base_idx + 1
        this->d1_motor_cmd.q_des_knee[sdk_leg] = command->motor_command.q[base_idx +

        // 目标速度映射
        this->d1_motor_cmd.qd_des_abad[sdk_leg] = command->motor_command.dq[base_idx
        this->d1_motor_cmd.qd_des_hip[sdk_leg] = command->motor_command.dq[base_idx +
        this->d1_motor_cmd.qd_des_knee[sdk_leg] = command->motor_command.dq[base_idx
```

```
        // PD增益映射
        this->d1_motor_cmd.kp_abad[sdk_leg] = command->motor_command.kp[base_idx + 0
        this->d1_motor_cmd.kp_hip[sdk_leg] = command->motor_command.kp[base_idx + 1]
        this->d1_motor_cmd.kp_knee[sdk_leg] = command->motor_command.kp[base_idx + 2

        this->d1_motor_cmd.kd_abad[sdk_leg] = command->motor_command.kd[base_idx + 0
        this->d1_motor_cmd.kd_hip[sdk_leg] = command->motor_command.kd[base_idx + 1]
        this->d1_motor_cmd.kd_knee[sdk_leg] = command->motor_command.kd[base_idx + 2

        // 前馈力矩映射
        this->d1_motor_cmd.tau_abad_ff[sdk_leg] = command->motor_command.tau[base_id
        this->d1_motor_cmd.tau_hip_ff[sdk_leg] = command->motor_command.tau[base_idx
        this->d1_motor_cmd.tau_knee_ff[sdk_leg] = command->motor_command.tau[base_id
    }

    // 发送命令到机器人
    int ret = this->d1_lowlevel.sendMotorCmd(this->d1_motor_cmd);
    if (ret < 0)
    {
        std::cout << LOGGER::WARNING << "Failed to send motor command" << std::endl;
    }
}
```

7.4 实机控制类完整实现

实机控制类是连接rl_sar框架和具体机器人硬件的桥梁，需要继承`RL`基类并实现三个纯虚函数：`Forward()`、`GetState()`和`SetCommand()`。以下是D1机器人实机控制类的头文件定义，展示了类的基本结构和成员变量。开发者在实现自己的机器人控制类时，可以参考这个模板进行修改，根据具体SDK的接口调整相应的成员变量和函数声明。

```
// rl_real_d1.hpp - D1实机控制头文件
#ifndef RL_REAL_D1_HPP
#define RL_REAL_D1_HPP

#include "rl_sdk.hpp"
#include "observation_buffer.hpp"
#include "inference_runtime.hpp"
#include "loop.hpp"
#include "fsm_d1.hpp"

// 引入机器人SDK头文件
#include "zsl-1/lowlevel.h"
#include "zsl-1/highlevel.h"

class RL_Real : public RL
{
public:
    RL_Real(int argc, char **argv);
    ~RL_Real();

private:
    // 必须实现的纯虚函数
```

```
    std::vector<float> Forward() override;
    void GetState(RobotState<float> *state) override;
    void SetCommand(const RobotCommand<float> *command) override;

    // 控制循环函数
    void RunModel();
    void RobotControl();

    // 控制循环对象
    std::shared_ptr<LoopFunc> loop_keyboard;
    std::shared_ptr<LoopFunc> loop_control;
    std::shared_ptr<LoopFunc> loop_rl;

    // 机器人SDK接口
    mc_sdk::LowLevel d1_lowlevel;
    mc_sdk::zsl_1::HighLevel d1_highlevel;
    mc_sdk::motorCmd d1_motor_cmd;
    mc_sdk::motorState* d1_motor_state;

    // 控制模式标志
    bool lowlevel_mode = false;

    // 模式切换函数
    void SwitchToLowLevel();
    void SwitchToHighLevel();
};

#endif // RL_REAL_D1_HPP
```

构造函数是控制类初始化的核心，需要完成配置读取、SDK初始化、网络连接、FSM加载和控制循环启动等一系列操作。D1机器人使用UDP网络通信，需要配置本地IP和机器人IP地址。在实际部署中，建议通过命令行参数传入网络配置，以便在不同网络环境下灵活使用。连接建立后，应该检查连接状态并打印相关信息，便于调试和问题排查。

```
// rl_real_d1.cpp - D1实机控制构造函数实现
RL_Real::RL_Real(int argc, char **argv)
{
    // 1. 设置角速度坐标系 - D1的角速度在世界坐标系中
    this->ang_vel_axis = "world";

    // 2. 读取机器人基础配置
    this->robot_name = "d1";
    this->ReadYaml(this->robot_name, "base.yaml");

    // 3. 自动加载FSM状态机
    if (FSMManager::GetInstance().IsTypeSupported(this->robot_name))
    {
        auto fsm_ptr = FSMManager::GetInstance().CreateFSM(this->robot_name, this);
        if (fsm_ptr)
        {
            this->fsm = *fsm_ptr;
        }
    }
    else
    {
        std::cout << LOGGER::ERROR << "[FSM] No FSM registered for robot: "
```

```
                << this->robot_name << std::endl;
    }

    // 4. 网络配置 - 支持命令行参数覆盖默认值
    int local_port = 43988;
    std::string local_ip = "192.168.234.2";   // 计算机IP
    std::string robot_ip = "192.168.234.1";   // D1默认IP

    if (argc >= 2) local_ip = argv[1];
    if (argc >= 3) robot_ip = argv[2];

    std::cout << LOGGER::INFO << "Connecting to D1 robot at " << robot_ip
              << " from " << local_ip << std::endl;

    // 5. 初始化SDK连接
    this->d1_lowlevel.initRobot(local_ip, local_port, robot_ip);
    this->d1_highlevel.initRobot(local_ip, local_port + 1, robot_ip);

    // 6. 等待连接建立（带超时机制）
    int retry_count = 0;
    while (!this->d1_lowlevel.checkConnect() && retry_count < 50)
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        retry_count++;
    }

    if (!this->d1_lowlevel.checkConnect())
    {
        std::cout << LOGGER::ERROR << "Failed to connect to D1 robot!" << std::endl;
    }
    else
    {
        std::cout << LOGGER::INFO << "Connected to D1 robot successfully!" << std::en
        // 打印电池电量信息
        uint32_t battery = this->d1_highlevel.getBatteryPower();
        std::cout << LOGGER::INFO << "Battery power: " << battery << "%" << std::endl
    }

    // 7. 初始化控制相关变量
    this->InitJointNum(this->params.Get<int>("num_of_dofs"));
    this->InitOutputs();
    this->InitControl();

    // 8. 启动控制循环
    this->loop_keyboard = std::make_shared<LoopFunc>("loop_keyboard", 0.05,
        std::bind(&RL_Real::KeyboardInterface, this));
    this->loop_control = std::make_shared<LoopFunc>("loop_control",
        this->params.Get<float>("dt"),
        std::bind(&RL_Real::RobotControl, this));
    this->loop_rl = std::make_shared<LoopFunc>("loop_rl",
        this->params.Get<float>("dt") * this->params.Get<int>("decimation"),
        std::bind(&RL_Real::RunModel, this));

    this->loop_keyboard->start();
    this->loop_control->start();
    this->loop_rl->start();
}
```

析构函数需要确保机器人在程序退出时能够安全停止，这是实机部署中非常重要的安全机制。D1机器人的析构函数会发送零力矩命令，使机器人进入被动模式，防止程序异常退出时机器人失控。建议在析构函数中多次发送停止命令，以确保命令能够被机器人正确接收和执行。

```
RL_Real::~RL_Real()
{
    // 发送零命令使机器人进入被动模式
    mc_sdk::motorCmd zero_cmd;
    for (int i = 0; i < 4; i++)
    {
        zero_cmd.kp_abad[i] = 0.0f;
        zero_cmd.kp_hip[i] = 0.0f;
        zero_cmd.kp_knee[i] = 0.0f;
        zero_cmd.kd_abad[i] = 3.0f;  // 保留阻尼以平稳停止
        zero_cmd.kd_hip[i] = 3.0f;
        zero_cmd.kd_knee[i] = 3.0f;
    }

    // 多次发送以确保命令被接收
    for (int i = 0; i < 100; i++)
    {
        this->d1_lowlevel.sendMotorCmd(zero_cmd);
        std::this_thread::sleep_for(std::chrono::milliseconds(2));
    }

    // 停止所有控制循环
    this->loop_keyboard->shutdown();
    this->loop_control->shutdown();
    this->loop_rl->shutdown();

    std::cout << LOGGER::INFO << "RL_Real exit" << std::endl;
}
```

7.5 FSM状态机实现

FSM状态机负责管理机器人的运行状态，包括被动模式、起身、行走、趴下等。每个状态都需要实现`Enter()`、`Run()`、`Exit()`和`CheckChange()`四个函数。以D1机器人为例，其FSM包含四个基本状态，状态之间的转换由用户输入触发。在实现自定义状态机时，需要特别注意状态转换的安全性，避免在不安全的状态下进行危险的操作。

```
// fsm_d1.hpp - D1机器人FSM状态机实现
#ifndef D1_FSM_HPP
#define D1_FSM_HPP

#include "fsm.hpp"
#include "rl_sdk.hpp"

namespace d1_fsm
{
```

```
// 被动状态 - 机器人可以被自由移动
class RLFSMStatePassive : public RLFSMState
{
public:
    RLFSMStatePassive(RL *rl) : RLFSMState(*rl, "RLFSMStatePassive") {}

    void Enter() override
    {
        std::cout << LOGGER::NOTE << "Entered passive mode. "
                  << "Press '0' or 'A' to switch to GetUp." << std::endl;
    }

    void Run() override
    {
        // 被动模式：kp=0（无位置控制），kd=8（仅阻尼）
        for (int i = 0; i < rl.params.Get<int>("num_of_dofs"); ++i)
        {
            fsm_command->motor_command.dq[i] = 0;
            fsm_command->motor_command.kp[i] = 0;
            fsm_command->motor_command.kd[i] = 8;
            fsm_command->motor_command.tau[i] = 0;
        }
    }

    void Exit() override{}

    std::string CheckChange() override
    {
        if (rl.control.current_keyboard == Input::Keyboard::Num0 ||
            rl.control.current_gamepad == Input::Gamepad::A)
        {
            return "RLFSMStateGetUp";
        }
        return state_name_;
    }
};
```

起身状态（GetUp）是机器人从被动模式过渡到站立姿态的关键状态。D1机器人的起身过程分为两个阶段：首先从当前姿态插值到预备姿态（蹲伏姿态），然后从预备姿态插值到默认站立姿态。这种两阶段设计可以避免机器人在起身过程中产生过大的关节运动，提高起身的稳定性和安全性。

```
// 起身状态 - 从被动模式过渡到站立姿态
class RLFSMStateGetUp : public RLFSMState
{
public:
    RLFSMStateGetUp(RL *rl) : RLFSMState(*rl, "RLFSMStateGetUp") {}

    float percent_pre_getup = 0.0f;
    float percent_getup = 0.0f;
    // D1预备姿态（蹲伏姿态）
    std::vector<float> pre_running_pos = {
        0.00, 1.36, -2.65,
        0.00, 1.36, -2.65,
        0.00, 1.36, -2.65,
        0.00, 1.36, -2.65
```

```
    };
    bool stand_from_passive = true;

    void Enter() override
    {
        percent_pre_getup = 0.0f;
        percent_getup = 0.0f;
        // 判断是否从被动模式起身
        if (rl.fsm.previous_state_->GetStateName() == "RLFSMStatePassive")
        {
            stand_from_passive = true;
        }
        else
        {
            stand_from_passive = false;
        }
        rl.now_state = *fsm_state;
        rl.start_state = rl.now_state;
    }

    void Run() override
    {
        if (stand_from_passive)
        {
            // 两阶段起身：先到预备姿态，再到站立姿态
            if (Interpolate(percent_pre_getup, rl.now_state.motor_state.q,
                pre_running_pos, 1.0f, "Pre Getting up", true)) return;
            if (Interpolate(percent_getup, pre_running_pos,
                rl.params.Get<std::vector<float>>("default_dof_pos"),
                2.0f, "Getting up", true)) return;
        }
        else
        {
            // 直接从当前位置到站立姿态
            if (Interpolate(percent_getup, rl.now_state.motor_state.q,
                rl.params.Get<std::vector<float>>("default_dof_pos"),
                1.0f, "Getting up", true)) return;
        }
    }

    void Exit() override{}

    std::string CheckChange() override
    {
        if (rl.control.current_keyboard == Input::Keyboard::P ||
            rl.control.current_gamepad == Input::Gamepad::LB_X)
        {
            return "RLFSMStatePassive";
        }
        if (percent_getup >= 1.0f)
        {
            if (rl.control.current_keyboard == Input::Keyboard::Num1 ||
                rl.control.current_gamepad == Input::Gamepad::RB_DPadUp)
            {
                return "RLFSMStateRLLocomotion";
            }
            else if (rl.control.current_keyboard == Input::Keyboard::Num9 ||
                     rl.control.current_gamepad == Input::Gamepad::B)
            {
                return "RLFSMStateGetDown";
            }
        }
```

```
        return state_name_;
    }
};
```

RL行走状态（RLLocomotion）是机器人执行强化学习策略的核心状态。在进入该状态时，会加载指定的策略配置和模型文件，初始化RL控制器。在运行过程中，会持续调用`RLControl()`函数执行神经网络推理和控制命令计算。该状态还支持在运行时切换不同的策略配置，实现多技能切换功能。

```
// RL行走状态 - 执行强化学习策略
class RLFSMStateRLLocomotion : public RLFSMState
{
public:
    RLFSMStateRLLocomotion(RL *rl) : RLFSMState(*rl, "RLFSMStateRLLocomotion") {}

    float percent_transition = 0.0f;

    void Enter() override
    {
        percent_transition = 0.0f;
        rl.episode_length_buf = 0;

        // 加载策略配置
        rl.config_name = "robot_lab";
        std::string robot_config_path = rl.robot_name + "/" + rl.config_name;
        try
        {
            rl.InitRL(robot_config_path);
            rl.now_state = *fsm_state;
        }
        catch (const std::exception& e)
        {
            std::cout << LOGGER::ERROR << "InitRL() failed: " << e.what() << std::en
            rl.rl_init_done = false;
            rl.fsm.RequestStateChange("RLFSMStatePassive");
        }
    }

    void Run() override
    {
        if (!rl.rl_init_done) rl.rl_init_done = true;

        // 显示当前控制命令
        std::cout << "\r\033[K" << std::flush << LOGGER::INFO
                  << "RL Controller [" << rl.config_name << "] "
                  << "x:" << rl.control.x
                  << " y:" << rl.control.y
                  << " yaw:" << rl.control.yaw << std::flush;

        // 执行RL控制
        RLControl();
    }

    void Exit() override
    {
        rl.rl_init_done = false;
    }
```

```
    std::string CheckChange() override
    {
        if (rl.control.current_keyboard == Input::Keyboard::P ||
            rl.control.current_gamepad == Input::Gamepad::LB_X)
        {
            return "RLFSMStatePassive";
        }
        else if (rl.control.current_keyboard == Input::Keyboard::Num9 ||
                 rl.control.current_gamepad == Input::Gamepad::B)
        {
            return "RLFSMStateGetDown";
        }
        return state_name_;
    }
};

} // namespace d1_fsm
```

7.6 FSM工厂注册

为了支持框架的自动FSM加载机制，需要创建FSM工厂类并使用宏进行注册。工厂类负责根据状态名称创建对应的状态对象，并提供机器人类型标识和支持的状态列表。注册完成后，还需要在`fsm_all.hpp`文件中添加新机器人的FSM头文件包含，这样框架才能在编译时将新的FSM工厂纳入管理。

```
// FSM工厂类定义
class D1FSMFactory : public FSMFactory
{
public:
    D1FSMFactory(const std::string& initial) : initial_state_(initial) {}

    std::shared_ptr<FSMState> CreateState(void *context, const std::string &state_nar
    {
        RL *rl = static_cast<RL *>(context);
        if (state_name == "RLFSMStatePassive")
            return std::make_shared<d1_fsm::RLFSMStatePassive>(rl);
        else if (state_name == "RLFSMStateGetUp")
            return std::make_shared<d1_fsm::RLFSMStateGetUp>(rl);
        else if (state_name == "RLFSMStateGetDown")
            return std::make_shared<d1_fsm::RLFSMStateGetDown>(rl);
        else if (state_name == "RLFSMStateRLLocomotion")
            return std::make_shared<d1_fsm::RLFSMStateRLLocomotion>(rl);
        return nullptr;
    }

    std::string GetType() const override{ return "d1"; }

    std::vector<std::string> GetSupportedStates() const override
    {
        return {
            "RLFSMStatePassive",
            "RLFSMStateGetUp",
            "RLFSMStateGetDown",
```

```
            "RLFSMStateRLLocomotion"
        };
    }

    std::string GetInitialState() const override{ return initial_state_; }

private:
    std::string initial_state_;
};

// 使用宏注册FSM工厂，指定初始状态为Passive
REGISTER_FSM_FACTORY(D1FSMFactory, "RLFSMStatePassive")

#endif // D1_FSM_HPP
```

在`fsm_all.hpp`中添加新机器人的FSM头文件：

```
// fsm_all.hpp - 更新包含列表
#ifndef FSM_ALL_HPP
#define FSM_ALL_HPP

#include "fsm_a1.hpp"
#include "fsm_b2.hpp"
#include "fsm_d1.hpp"      // 添加D1机器人FSM
#include "fsm_g1.hpp"
#include "fsm_go2.hpp"
// ... 其他机器人FSM

#endif // FSM_ALL_HPP
```

7.7 二次开发注意事项与调试技巧

在进行二次开发时，有一些常见的问题和注意事项需要特别关注。首先是坐标系的一致性问题，不同的机器人SDK可能使用不同的坐标系约定，包括IMU数据的坐标系、四元数的顺序（wxyz或xyzw）、角速度是在世界坐标系还是机体坐标系中表示等。这些差异如果处理不当，会导致策略行为完全错误。建议在开发初期通过打印日志的方式验证各项数据的正确性。

常见问题排查清单：

问题现象	可能原因	解决方案
机器人腿部运动混乱	关节映射错误	检查SDK_LEG_MAP映射表
机器人原地打转	角速度坐标系错误	检查ang_vel_axis设置
机器人倒向一侧	四元数顺序错误	确认四元数为(w,x,y,z)顺序
策略输出异常	观测顺序不一致	核对config.yaml中observations顺序
关节抖动严重	PD增益设置不当	调整rl_kp和rl_kd参数

问题现象	可能原因	解决方案
力矩超限报警	torque_limits设置过小	根据电机规格调整限制值

调试建议：

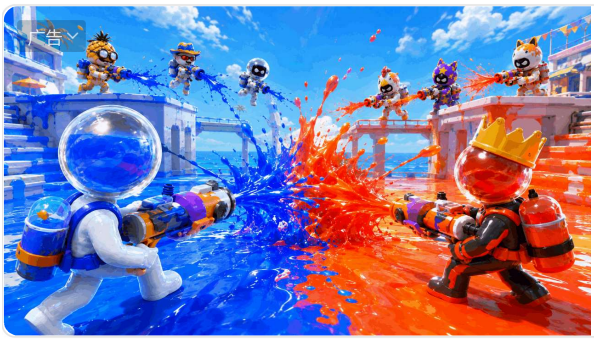
1. **分阶段测试**：先在仿真环境中验证配置正确性，再进行实机测试。首次实机测试时务必将机器人吊起，避免摔倒损坏。
2. **启用日志功能**：在头文件中定义`CSV_LOGGER`宏可以启用CSV日志记录功能，记录关节位置、速度、力矩等数据，便于离线分析问题。
3. **使用绘图功能**：定义`PLOT`宏可以启用实时绘图功能，直观观察目标位置和实际位置的跟踪情况，快速定位控制问题。
4. **渐进式调试**：先测试被动模式和起身功能，确保基本控制正常后再测试RL行走功能。

下图是MuJoCo仿真的效果，使用`./cmake_build/bin/rl_sim_mujoco d1 scene`启动仿真后，按0导入状态机进入被动模式，然后按R重置机器人位置，最后按1进入RL控制模式开始行走。

下图是Gazebo仿真的效果。首先使用`ros2 launch rl_sar gazebo.launch.py rname:=d1`启动Gazebo仿真环境和机器人模型，然后在另一个终端中运行`ros2 run rl_sar rl_sim`启动RL控制节点。控制节点启动后，按0键导入状态机进入被动模式，按R键重置机器人位置和姿态，最后按1键进入RL控制模式开始行走。Gazebo仿真相比MuJoCo提供了更真实的物理引擎和传感器模拟，适合进行更接近实机的验证测试。

8. 总结与展望

本文深入介绍了rl_sar框架的架构设计、核心模块功能以及部署流程。作为一个专门为四足机器人、轮足机器人和人形机器人设计的强化学习部署框架，rl_sar通过分层架构和模块化设计，有效地解决了从仿真到实机部署过程中的各种挑战。框架的核心优势在于其统一的接口设计，使得开发者可以使用相同的代码在Gazebo、MuJoCo等仿真环境和真实机器人之间无缝切换，大大降低了Sim-to-Real的开发成本。



色彩争夺战！占领更多地盘
极限枪战大乱斗
进入小游戏

框架的主要特点包括：支持多种推理后端（TorchScript和ONNX），适应不同的部署场景；灵活的配置系统，通过YAML文件管理机器人参数和策略配置；完善的有限状态机设计，确保机器人在不同状态下安全运行；以及内置的安全保护机制，防止异常情况造成硬件损坏。通过本文介绍的二次开发方法，开发者可以方便地将框架扩展到自己的机器人平台，只需实现三个核心接口函数即可完成新机器人的集成。

9. 参考资料

- rl_sar GitHub仓库：https://github.com/fan-ziqi/rl_sar
- robot_lab（IsaacLab训练框架）：https://github.com/fan-ziqi/robot_lab
- AgiBot D1 SDK文档：https://github.com/AgibotTech/agibot_D1_Edu-Ultra
- PyTorch TorchScript文档：https://pytorch.org/docs/stable/jit.html
- ONNX Runtime文档：https://onnxruntime.ai/docs/

👉 关注我们，发现更多有深度的自动驾驶/具身智能/GitHub 内容！

敢敢AUTOHUB
致力于分享一线具身智能/自动驾驶相关知识，希望能够一起不断学习成长
420篇原创内容

公众号

🚀 往期内容回顾 👀

📄🔥 读读代码 | DINOv3在语义分割中的应用：从遥感影像到地物识别的完整实战指南
📄🔥 行业杂谈 | Claude Skills：AI 能力扩展的革命性架构
📄🔥 智元D1开发实战 | 一篇搞定——机器人配置训练调参全解析！

喜欢作者

智元D1开发实战 · 目录 ≡

< 上一篇
智元D1开发实战 | 这一篇就够——基于Robot Lab的强化学习

下一篇 >
智元D1开发教程 | Sim2Real全指南（上）——从控制接入到真机落地

阅读 188



WorkBuddy AI办公助手，免部署安装即用

WorkBuddy

查看